Name	Last commit message	Last commit date
..		
img	Картинки распределены по папкам	2 years ago
README.md	Исправлены ссылки на картинки	last year

README.md

Логистическая регрессия

Цель работы

Познакомиться с широко используемым методом бинарной классификации - логистической регрессией.

Содержание работы

1. Сгенерировать матрицу признаков и вектор целей для задачи классификации с использованием `make_classification` из библиотеки `sklearn.datasets`. Число классов возьмите равным двум.
2. Реализовать модель логистической регрессии методом градиентного спуска, не используя библиотечные функции.
3. Оценить качество построенной модели, используя метрики accuracy и F1-score.
4. Реализовать модель логистической регрессии `LogisticRegression` из библиотеки `sklearn.linear_model` и оценить качество построенной модели, используя метрики accuracy и F1-score.
5. Сравнить результаты двух реализаций.

Методические указания

Загрузим необходимые библиотеки:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Сгенерируем матрицу признаков и вектор целей для задачи классификации. Для этого воспользуемся библиотечной функцией make_classification, которая генерирует случайное распределение по заданным параметрам. Число генерируемых точек данных задачим равным 1000, количество признаков - 2 для того, чтобы их можно было изобразить на графике. Оба признака будут информативными, то есть влиять на значение целевой переменной. Классов у нас тоже будет два, то есть будем решать задачу бинарной классификации. Параметр class_sep задает расстояние (зазор) между классами.

```
from sklearn.datasets import make_classification
X,y = make_classification (n_samples=1000,
                           n_features=2,
                           n_informative=2,
                           n_redundant=0,
                           n_classes=2,
                           class_sep=2,
                           random_state=1)
```

Выведем первые пять строк X:

```
pd.DataFrame(X).head()
```

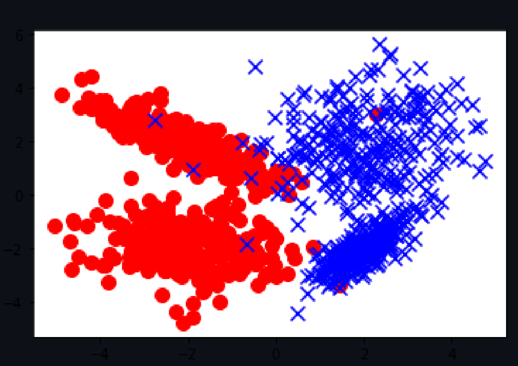
Вот что нужно увидеть в результате.

index	0	1
0	1.8634113686607574	0.08764555156962572
1	-1.530997171022319	1.9011824126143653
2	-0.017224044544175365	-2.591111592515568
3	-2.0320385262642717	-1.8011621832132958
4	4.620554804681561	0.9011235462980076

Анализировать такие данные в табличном виде не очень наглядно, поэтому построим их визуальное распределение. Обратите внимание, как в данном примере используется условная индексация одного массива (признаков) другим массивом (целевой переменной):

```
plt.scatter(X[:, 0][y==0], X[:, 1][y==0], marker="o", c='r', s=100)
plt.scatter(X[:, 0][y==1], X[:, 1][y==1], marker="x", c='b', s=100)
plt.show()
```

В результате должна получиться такая визуализация:



Теперь приступим к реализации метода логистической регрессии своими руками. Сейчас используем немного другой подход, чем в работе, посвященной линейной регрессии. Так как мы уже знаем два метода машинного обучения, то можем заметить, что в большой степени они похожи, так что некоторый код у нас будет повторяться. Но в деталях реализации они будут различаться. Мы создадим два класса: первый реализует базовый функционал гралиентного спуска, который будет общий для всех моделей, основанных на нем. Второй класс - моделеспецифичный, в нем пропишем все, что касается именно логистической регрессии. Как всегда будем придерживаться соглашений об именовании методов, которые приняты в библиотеке sklearn.

Создадим конструктор класса, реализующего градиентный спуск. Он будет инициализировать необходимые параметры для градиентного спуска. Обратите внимание, что это не параметры модели, так как их количество зависит от размерности исходных данных, а мы хотим написать универсальный класс:

```
class SGD():
    def __init__(self, alpha=0.5, n_iters=1000):
        self.b = None
        self._alpha = alpha
        self._n_iters = n_iters
```

Теперь реализуем один шаг градиентного спуска. Аргументом этого метода сделаем градиент функции ошибки (так как он специфичен для модели):

```
    def gradient_step(self, b, b_grad):
        return b - self._alpha * b_grad
```

Следующий шаг - реализация всего алгоритма оптимизации методом градиентного спуска:

```
    def optimize(self, X, y, start_b, n_iters):
        b = start_b.copy()
        for i in range(n_iters):
            b_grad = self.grad_func(X, y, b)
            b = self.gradient_step(b, b_grad)
        return b
```

И, наконец, последняя обертка - метод обучения, который как раз и принимает на вход точки данных датасета:

```
    def fit(self, X, y):
        m = X.shape[1]
        start_b = np.ones(m)
        self.b = self.optimize(X, y, start_b, self._n_iters)
```

Обратите внимание, что в этом классе мы не реализовали использованный в одном месте метод grad_func(X, y, b). Этот метод моделеспецифичный, поэтому определим его во втором классе. Такая архитектура, в частности, означает, что наш класс SGD не можут использоваться сам по себе - он абстрактный. В конкретной реализации этого класса должен быть определен этот метод.

Создадим класс, реализующего логистическую регрессию. Конструктор здесь нам не нужен, так как он будет наследоваться. А начнем непосредственно с функции гипотезы:

```
class LogReg(SGD):
    def sigmoid(self, X, b):
        return 1. / (1. + np.exp(-X.dot(b)))
```

Теперь нужно определить тот самый метод grad_func(X, y, b), то есть метод вычисления градиента (частных производных функции ошибки):

```
    def grad_func(self, X, y, b):
        n = X.shape[0]
        grad = 1. / n * X.transpose().dot(self.sigmoid(X, b) - y)
        return grad
```

Теперь методы предсказания. Здесь у нас их будет два. Первый выдает значения сигмоиды - вероятность отнесения данного объекта к полоительному классу:

```
    def predict_proba(self, X):
        return self.sigmoid(X, self.b)
```

А второй - это точное предсказание:

```
    def predict(self, X):
        y_pred = self.predict_proba(X) > 0.5
        return y_pred
```

Такая архитектура классов гораздо более универсальная и удобная, чем та, которую мы использовали в предыдущих работах. Она очень подобна той, которая используется в самой библиотеке sklearn.

Но мы продолжаем, и теперь нам нужно использовать созданную модель. Создаём экземпляр класса модели:

```
logreg = LogReg()
```

Добавим фиктивный столбец единиц к матрице признаков X:

```
X = np.hstack([np.ones(X.shape[0])[:, np.newaxis], X])
```

Обучим модель и сделаем предсказание:

```
logreg.fit(X, y)
y_pred = logreg.predict(X)
```

Выводим метрики качества:

```
from sklearn.metrics import accuracy_score, f1_score
ac = accuracy_score(y, y_pred)
f1 = f1_score(y, y_pred)
print(f'accuracy = {ac:.2f} F1-score = {f1:.2f}')
```

Выведем значения вероятностей для каждого объекта принадлежать тому или иному классу:

```
y_pred_proba = logreg.predict_proba(X_test)
```

После создания модели логистической регрессии логичным шагом будет вывести ее на график вместе с точками данных. Проблема в том, что это не так просто, как в случае с линейной регрессией, так как мы имеем два измерения признаков плюс еще значение самой функции модели. Для того, чтобы наглядно увидеть, как сочетается значение модели с точками воспользуемся построением контурного графика.

Для начала надо подготовить равномерные данные для рисования функции гипотезы. Нам понадобится создать двумерную сетку. К счастью, в numpy есть необходимые элементы. Подробный разбор кода выходит за рамки данного пособия, так как использует продвинутые возможности библиотеки numpy. Если вам интересно, как работает этот код, обратитесь к документации к используемым методам:

```
xx, yy = np.meshgrid(
    np.arange(X.min(axis=0)[1]-1, X.max(axis=0)[1]+1, 0.01),
    np.arange(X.min(axis=0)[2]-1, X.max(axis=0)[2]+1, 0.01))
XX = np.array(list(zip(xx.ravel(), yy.ravel()))).reshape((-1, 2))
XX = np.array([(1, *xx) for xx in XX])
```

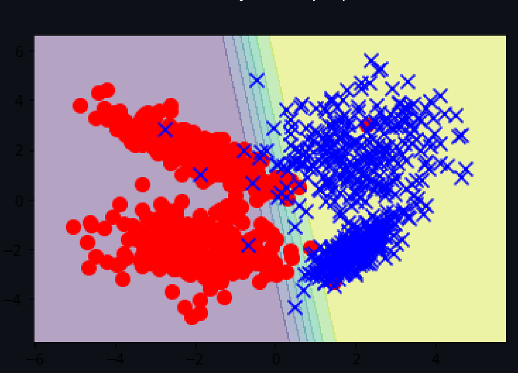
В данном коде мы создаем двумерную матрицу, содержащую все комбинации значений признаков в заданном диапазоне. Другими словами, мы создаем равномерную сетку в прямоугольнике от минимального до максимального значения каждого признака (отступая для красоты 1 в обоих направлениях). Попробуйте вывести получившиеся переменные, чтобы понять принцип построения данной сетки. А после мы используем матрицу XX как исходные данные для модели:

```
Z = logreg.predict_proba(XX)
Z = Z.reshape(xx.shape)
```

Данный код выполнит предсказание модели в каждой точке нашей сетки. Эти данные мы сможем использовать для того, чтобы построить контурный график вот так:

```
plt.contourf(xx, yy, Z, alpha=0.4)
plt.scatter(X[:, 1][y==0], X[:, 2][y==0], marker="o", c='r', s=100)
plt.scatter(X[:, 1][y==1], X[:, 2][y==1], marker="x", c='b', s=100)
```

В итоге мы должны получить график, похожий на следующий рисунок:



Сделайте вывод по данному графику. Насколько хорошая получилась модель для имеющихся данных?

Теперь проделаем то же самое, используя библиотечные функции.

Создадим экземпляр класса:

```
model = LogisticRegression()
```

Обучим модель и сделаем предсказание:

```
model.fit(X, y)
y_pred_lr = model.predict(X)
```

Далее необходимо вывести метрики качества аналогичным образом и сделать сравнение результатов.

Задания для самостоятельного выполнения

1. Выведите результаты работы библиотечной модели - в численном и в графическом виде.
2. Проверьте работу модели с другими значениями скорости обучения. Найдите значение, при котором градиентный спуск расходится.
3. Модифицируйте код модели таким образом, чтобы фиктивный столбец единиц добавлялся к матрице признаков внутри класса.
4. Поэкспериментируйте с разными значениями параметра class_sep при генерации датасета. Визуализируйте полученные распределения. Сделайте вывод о том, как этот параметр влияет на точность получаемых моделей.
5. Сгенерируйте датасет с большим числом признаков и примените к нему созданную модель.
6. Сгенерируйте датасет с большим количеством классов и реализуйте в классе алгоритм "один против всех". Решите задачу множественной классификации средствами sklearn.
7. Выведите значения вероятностей для каждого объекта принадлежать тому или иному классу для библиотечной модели `LogisticRegression`.

Контрольные вопросы

1. Сформулируйте, в чем состоит задача классификации, придумайте несколько примеров.
2. Что такое шаг градиентного спуска?
3. Какая функция используется в качестве функции ошибки в модели логистической регрессии?
4. Зачем при реализации логистической регрессии к матрице признаков добавлялся столбец из единиц?

Дополнительные задания:

1. На основе классов, написанных в этой работе, создайте класс, реализующий модель линейной регрессии, но уже в новой архитектуре. В базовом классе реализуйте мезанизм адаптивной скорости обучения, нормализации входных данных, остановки при стабилизации функции ошибки.
2. Постройте ROC кривую и найдите площадь под этой кривой, используя функции `roc_curve`, `roc_auc_score` из библиотеки `sklearn.metrics`. Оцените качество модели по этой кривой.